

Examen Fundamentos de la Especialidad II

Apellidos-Nombres: Aldás García Joel Edilberto

Carrera: Ingeniería en Informática

Docente: MSc. Viviana Flores

Evaluación: Examen Parcial I

Sección A – Ingeniería de Software

Ejercicio 1

Implemente RackRateStrategy (temporada alta: \$80/noche + recargo 20%), LowSeasonRateStrategy (temporada baja: \$80/noche con 15% de descuento) y CorporateRateStrategy (\$60/noche fijo). Implemente ReservationFactory con un método estático CreateReservation(...) que construya la reserva y devuelva la estrategia correspondiente, lanzando una excepción controlada si RateType no es válido.

RackRateStrategy.cs

```
Patterns > Strategy > RackRateStrategy.cs
1 namespace HotelRealQuito.Strategy;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5
6 // Temporada alta: $80 por noche mas 20% de recargo
7 public class RackRateStrategy : IRateStrategy
8 {
9     private const double PricePerNight = 80.0;
10    private const double Surcharge = 1.2; // 20% de recargo
11
12    public double CalculateCost(Reservation reservation)
13    {
14        double cost = PricePerNight * Surcharge * reservation.Nights;
15        Console.WriteLine($"[Tarifa Alta] ${PricePerNight} x {reservation.Nights} noches + 20% recargo = ${cost:F2}");
16        return cost;
17    }
18 }
```

LowSeasonRateStrategy.cs

```
Patterns > Strategy > LowSeasonRateStrategy.cs
1 namespace HotelRealQuito.Strategy;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5
6 // Temporada baja: $80 por noche con 15% de descuento
7 public class LowSeasonRateStrategy : IRateStrategy
8 {
9     private const double PricePerNight = 80.0;
10    private const double Discount = 0.85; // 15% de descuento
11
12    public double CalculateCost(Reservation reservation)
13    {
14        double cost = PricePerNight * Discount * reservation.Nights;
15        Console.WriteLine($"[Tarifa Baja] ${PricePerNight} x {reservation.Nights} noches - 15% descuento = ${cost:F2}");
16        return cost;
17    }
18 }
19 }
```

CorporateRateStrategy.cs

```
Patterns > Strategy > CorporateRateStrategy.cs
1 namespace HotelRealQuito.Strategy;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5
6 // Tarifa corporativa: $60 por noche fijo, sin recargos ni descuentos
7 public class CorporateRateStrategy : IRateStrategy
8 {
9     private const double PricePerNight = 60.0;
10
11     public double CalculateCost(Reservation reservation)
12     {
13         double cost = PricePerNight * reservation.Nights;
14         Console.WriteLine($"[Tarifa Corporativa] ${PricePerNight} x {reservation.Nights} noches = ${cost:F2}");
15         return cost;
16     }
17 }
18
```

ReservationFactory.cs

```
Reservation.cs  ReservationFactory.cs X  ReservationRepository.cs  ReservationService.cs  Program.cs  CorporateRateStra
Patterns > Factory > ReservationFactory.cs
1 namespace HotelRealQuito.Factory;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5 using HotelRealQuito.Strategy;
6
7 // Fabrica que crea reservas y asigna la estrategia de tarifa correcta
8 public class ReservationFactory
9 {
10     // Crea una reserva y devuelve la estrategia que le corresponde
11     public static (Reservation reservation, IRateStrategy strategy) CreateReservation(
12         string id,
13         string guestName,
14         string roomType,
15         int nights,
16         string rateType)
17     {
18         // Crear la reserva con estado inicial Creada
19         Reservation reservation = new Reservation
20         {
21             Id = id,
22             GuestName = guestName,
23             RoomType = roomType,
24             Nights = nights,
25             RateType = rateType,
26             Status = ReservationStatus.Creada,
27             TotalCost = 0.0
28         };
29
30         // Asignar la estrategia segun el tipo de tarifa
31         IRateStrategy strategy = rateType.ToLower() switch
32         {
33             "temporadaalta" => new RackRateStrategy(),
34             "temporadabaja" => new LowSeasonRateStrategy(),
35             "corporativa" => new CorporateRateStrategy(),
36             _ => throw new ArgumentException(
37                 $"Tipo de tarifa no valido: '{rateType}'. " +
38                 $"Los tipos validos son: temporadaAlta, temporadaBaja, corporativa.");
39         };
40
41         Console.WriteLine(
42             $"[Fabrica] Reserva {id} creada para {guestName} " +
43             $"| Habitacion: {roomType} | Noches: {nights} " +
44             $"| Tarifa: {rateType}");
45
46         return (reservation, strategy);
47     }
48 }
49
```

Console

```
>> Ejercicio 1: Crear reservas con diferentes tarifas

[Fabrica] Reserva RES-001 creada para Laura Gomez | Habitacion: simple | Noches: 3 | Tarifa: temporadaAlta
[Tarifa Alta] $80 x 3 noches + 20% recargo = $288,00
[Repositorio] Reserva RES-001 guardada correctamente.
[Email] Notificacion enviada a Laura Gomez: su reserva RES-001 ha cambiado a estado Confirmada.
[SMS] Mensaje enviado a Laura Gomez: su reserva RES-001 ahora esta en estado Confirmada.

[Fabrica] Reserva RES-002 creada para Pedro Salas | Habitacion: doble | Noches: 2 | Tarifa: temporadabaja
[Tarifa Baja] $80 x 2 noches - 15% descuento = $136,00
[Repositorio] Reserva RES-002 guardada correctamente.
[Email] Notificacion enviada a Pedro Salas: su reserva RES-002 ha cambiado a estado Confirmada.
[SMS] Mensaje enviado a Pedro Salas: su reserva RES-002 ahora esta en estado Confirmada.

[Fabrica] Reserva RES-003 creada para Carmen Ruiz | Habitacion: suite | Noches: 5 | Tarifa: corporativa
[Tarifa Corporativa] $60 x 5 noches = $300,00
[Repositorio] Reserva RES-003 guardada correctamente.
[Email] Notificacion enviada a Carmen Ruiz: su reserva RES-003 ha cambiado a estado Confirmada.
[SMS] Mensaje enviado a Carmen Ruiz: su reserva RES-003 ahora esta en estado Confirmada.
```

Ejercicio 2

Implemente EmailReservationNotifier, SmsReservationNotifier y ReservationRepository (lista en memoria). En un método ProcessReservation, orqueste: crear la reserva con la Factory, calcular costo con la Strategy, guardarla con el Repository y notificar a los observadores el cambio a estado Confirmada. Adjunte captura de la ejecución mostrando los tres mensajes de salida.

EmailReservationNotifier.cs

```
Patterns > Observer > EmailReservationNotifier.cs > ...
1 namespace HotelRealQuito.Observer;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5
6 // Notificador que envia un email al huesped cuando cambia el estado
2 references
7 public class EmailReservationNotifier : IReservationObserver
8 {
9     2 references
10     public void OnStatusChanged(Reservation reservation, ReservationStatus newStatus)
11     {
12         Console.WriteLine(
13             $"[Email] Notificacion enviada a {reservation.GuestName}: " +
14             $"su reserva {reservation.Id} ha cambiado a estado {newStatus}.");
15     }
16 }
```

SmsReservationNotifier.cs

```
Patterns > Observer > SmsReservationNotifier.cs > {} HotelRealQuito.Observer
1 namespace HotelRealQuito.Observer;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5
6 // Notificador que envia un SMS al huesped cuando cambia el estado
2 references
7 public class SmsReservationNotifier : IReservationObserver
8 {
9     2 references
10     public void OnStatusChanged(Reservation reservation, ReservationStatus newStatus)
11     {
12         Console.WriteLine(
13             $"[SMS] Mensaje enviado a {reservation.GuestName}: " +
14             $"su reserva {reservation.Id} ahora esta en estado {newStatus}.");
15     }
16 }
```

ReservationFactory.cs

```
Patterns > Factory > ReservationFactory.cs > {} HotelRealQuito.Factory > ReservationFactory > CreateReservation(string, string)
1 namespace HotelRealQuito.Factory;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5 using HotelRealQuito.Strategy;
6
7 // Fabrica que crea reservas y asigna la estrategia de tarifa correcta
8 public class ReservationFactory
9 {
10     // Crea una reserva y devuelve la estrategia que le corresponde
11     public static (Reservation reservation, IRateStrategy strategy) CreateReservation(
12         string id,
13         string guestName,
14         string roomType,
15         int nights,
16         string rateType)
17     {
18         // Crear la reserva con estado inicial Creada
19         Reservation reservation = new Reservation
20         {
21             Id = id,
22             GuestName = guestName,
23             RoomType = roomType,
24             Nights = nights,
25             RateType = rateType,
26             Status = ReservationStatus.Creada,
27             TotalCost = 0.0
28         };
29
30         // Asignar la estrategia segun el tipo de tarifa
31         IRateStrategy strategy = rateType.ToLower() switch
32         {
33             "temporadaalta" => new RackRateStrategy(),
34             "temporadabaja" => new LowSeasonRateStrategy(),
35             "corporativa" => new CorporateRateStrategy(),
36             _ => throw new ArgumentException(
37                 $"Tipo de tarifa no valido: '{rateType}'. " +
38                 $"Los tipos validos son: temporadaAlta, temporadaBaja, corporativa.")
39         };
40
41         Console.WriteLine(
42             $"[Fabrica] Reserva {id} creada para {guestName} " +
43             $"| Habitacion: {roomType} | Noches: {nights} " +
44             $"| Tarifa: {rateType}");
45
46         return (reservation, strategy);
47     }
48 }
49
```

ReservationRepository.cs

```
Patterns > Repository > ReservationRepository.cs > {} HotelRealQuito.Repository > ReservationRepository
1  namespace HotelRealQuito.Repository;
2
3  using HotelRealQuito.Interfaces;
4  using HotelRealQuito.Models;
5
6  // Repositorio que guarda las reservas en memoria usando una lista
   2 references
7  public class ReservationRepository : IReservationRepository
8  {
   4 references
9      private readonly List<Reservation> _reservations = new List<Reservation>();
10
11     // Guarda una nueva reserva en la lista
   2 references
12     public void Save(Reservation reservation)
13     {
14         _reservations.Add(reservation);
15         Console.WriteLine($"[Repositorio] Reserva {reservation.Id} guardada correctamente.");
16     }
17
18     // Busca una reserva por su ID devuelve null si no existe
   1 reference
19     public Reservation? GetById(string id)
20     {
21         return _reservations.FirstOrDefault(r => r.Id == id);
22     }
23
24     // Devuelve todas las reservas guardadas
   3 references
25     public List<Reservation> GetAll()
26     {
27         return _reservations;
28     }
29
30     // Actualiza el estado de una reserva existente
   1 reference
31     public void UpdateStatus(string id, ReservationStatus newStatus)
32     {
33         Reservation? reservation = _reservations.FirstOrDefault(r => r.Id == id);
34         if (reservation != null)
35         {
36             reservation.Status = newStatus;
37             Console.WriteLine($"[Repositorio] Reserva {id} actualizada a estado: {newStatus}");
38         }
39     }
40 }
41
```

ReservationService.cs

```
Patterns > ReservationService.cs > {} HotelRealQuito.Patterns > ReservationService > ProcessReservation(string, string, string,
1 namespace HotelRealQuito.Patterns;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5 using HotelRealQuito.Factory;
6
7 // Servicios crear reserva, calcular costo, guardar y notificar
8 public class ReservationService
9 {
10     private readonly IReservationRepository _repository;
11     private readonly List<IReservationObserver> _observers = new List<IReservationObserver>();
12
13     // Recibe el repositorio por inyeccion de dependencias
14     public ReservationService(IReservationRepository repository)
15     {
16         _repository = repository;
17     }
18
19     // Agrega un observador para recibir notificaciones
20     public void Subscribe(IReservationObserver observer)
21     {
22         _observers.Add(observer);
23     }
24
25     // Quita un observador para que deje de recibir notificaciones
26     public void Unsubscribe(IReservationObserver observer)
27     {
28         _observers.Remove(observer);
29     }
30
31     // Avisar a todos los observadores que cambio el estado
32     private void NotifyObservers(Reservation reservation, ReservationStatus newStatus)
33     {
34         foreach (var observer in _observers)
35         {
36             observer.OnStatusChanged(reservation, newStatus);
37         }
38     }
39
40     // Metodo principal que orquesta todo el proceso de una reserva
41     public Reservation ProcessReservation(
42         string id,
43         string guestName,
44         string roomType,
45         int nights,
46         string rateType)
47     {
48         //Crear la reserva usando la Fabrica
49         var (reservation, strategy) = ReservationFactory.CreateReservation(
50             id, guestName, roomType, nights, rateType);
51
52         //Calcular el costo total con la estrategia correspondiente
53         reservation.TotalCost = strategy.CalculateCost(reservation);
54
55         //Guardar la reserva en el repositorio
56         _repository.Save(reservation);
57
58         //Cambiar el estado a Confirmada
59         reservation.Status = ReservationStatus.Confirmada;
60
61         //Notificar a los observadores del cambio de estado
62         NotifyObservers(reservation, ReservationStatus.Confirmada);
63
64         return reservation;
65     }
66 }
67
```

Console

>> Ejercicio 2: Verificar notificaciones y repositorio

Reservas guardadas: 3

[Resumen] ID: RES-001 | Huesped: Laura Gomez | Habitacion: simple | Noches: 3 | Tarifa: temporadaAlta | Total: \$288,00 | Estado: Confirmada
[Resumen] ID: RES-002 | Huesped: Pedro Salas | Habitacion: doble | Noches: 2 | Tarifa: temporadabaja | Total: \$136,00 | Estado: Confirmada
[Resumen] ID: RES-003 | Huesped: Carmen Ruiz | Habitacion: suite | Noches: 5 | Tarifa: corporativa | Total: \$300,00 | Estado: Confirmada

Ejercicio 3

Se entrega el siguiente fragmento con problemas de diseño:

```
public class HotelManager
{
    public void HandleReservation(string id, string guest, string rateType, int nights)
    {
        double cost = 0;
        if (rateType == "temporadaAlta") { cost = nights * 80 * 1.2; }
        else if (rateType == "temporadaBaja") { cost = nights * 80 * 0.85; }
        else if (rateType == "corporativa") { cost = nights * 60; }

        HotelDatabase.Reservations.Add(new Reservation { Id = id, GuestName = guest, TotalCost = cost });

        Console.WriteLine($"Email enviado a {guest}");
        Console.WriteLine($"SMS enviado a {guest}");
    }
}
```

Identifique las violaciones SOLID encontradas en el fragmento de código.

1. El método HandleReservation hace todo calcula costo, guarda en la base de datos y envia email y sms. **Violenta el principio de responsabilidad única.**

Solución aplicable: Strategy para calcular los costos-Respository para separar la persistencia de cada clase- Observer para separar la notificación en clases dependientes.

2. Tambien se puede observar que si se quiere agregar un nuevo método de pago o tarifa como tal se debe modificar todo el código agregando otro else if no asegurando que lo demás no se rompa. **Violenta el principio de abierto y cerrado.**

Solución aplicable: Strategy aquí podríamos agregar varias tarifas ya que se crea la clase IRateStrategy y solo se aumentaría una nueva clase con nuevas tarifas sin intervenir o modificar más código.

```
1 namespace HotelRealQuito.Analysis;
2
3 using HotelRealQuito.Interfaces;
4 using HotelRealQuito.Models;
5 using HotelRealQuito.Patterns;
6
7 1 reference
8 public class HotelManager
9 {
10     2 references
11     private readonly ReservationService _service;
12
13     0 references
14     public HotelManager(IReservationRepository repository)
15     {
16         _service = new ReservationService(repository);
17     }
18
19     // Ahora el metodo solo orquesta, delegando cada responsabilidad
20     0 references
21     public Reservation HandleReservation(string id, string guest, string rateType, int nights)
22     {
23         return _service.ProcessReservation(id, guest, "doble", nights, rateType);
24     }
25 }
```

Para refactorizar esta clase hacemos uso de Strategy + Repository + ReservationService.

Console

```
>> Ejercicio 1: Crear reservas con diferentes tarifas

[Fabrica] Reserva RES-001 creada para Laura Gomez | Habitacion: simple | Noches: 3 | Tarifa: temporadaAlta
[Tarifa Alta] $80 x 3 noches + 20% recargo = $288,00
[Repositorio] Reserva RES-001 guardada correctamente.
[Email] Notificacion enviada a Laura Gomez: su reserva RES-001 ha cambiado a estado Confirmada.
[SMS] Mensaje enviado a Laura Gomez: su reserva RES-001 ahora esta en estado Confirmada.

[Fabrica] Reserva RES-002 creada para Pedro Salas | Habitacion: doble | Noches: 2 | Tarifa: temporadabaja
[Tarifa Baja] $80 x 2 noches - 15% descuento = $136,00
[Repositorio] Reserva RES-002 guardada correctamente.
[Email] Notificacion enviada a Pedro Salas: su reserva RES-002 ha cambiado a estado Confirmada.
[SMS] Mensaje enviado a Pedro Salas: su reserva RES-002 ahora esta en estado Confirmada.

[Fabrica] Reserva RES-003 creada para Carmen Ruiz | Habitacion: suite | Noches: 5 | Tarifa: corporativa
[Tarifa Corporativa] $60 x 5 noches = $300,00
[Repositorio] Reserva RES-003 guardada correctamente.
[Email] Notificacion enviada a Carmen Ruiz: su reserva RES-003 ha cambiado a estado Confirmada.
[SMS] Mensaje enviado a Carmen Ruiz: su reserva RES-003 ahora esta en estado Confirmada.

>> Ejercicio 2: Verificar notificaciones y repositorio

Reservas guardadas: 3

[Resumen] ID: RES-001 | Huesped: Laura Gomez | Habitacion: simple | Noches: 3 | Tarifa: temporadaAlta | Total: $288,00 | Estado: Confirmada
[Resumen] ID: RES-002 | Huesped: Pedro Salas | Habitacion: doble | Noches: 2 | Tarifa: temporadabaja | Total: $136,00 | Estado: Confirmada
[Resumen] ID: RES-003 | Huesped: Carmen Ruiz | Habitacion: suite | Noches: 5 | Tarifa: corporativa | Total: $300,00 | Estado: Confirmada

>> Ejercicio 3: Probar excepcion con tarifa invalida

[Excepcion controlada] Tipo de tarifa no valido: 'estudiante'. Los tipos validos son: temporadaAlta, temporadaBaja, corporativa.

=====
=== Fin del Sistema de Reservas ===
```

ENLACE DEL REPOSITORIO EN GITHUB:

https://github.com/JOALU26/EXAMEN_FUNDAMENTOS_II.git

Sección B – Base De Datos II

Query	Query History
1	-- Crear la base de datos de la clinica veterinaria
2	CREATE DATABASE patitas_felices;

-CREACION DE TABLAS

1	--tabla dueños de mascotas
2	CREATE TABLE propietarios (
3	id_propietario SERIAL PRIMARY KEY,
4	nombre VARCHAR(100) NOT NULL,
5	telefono VARCHAR(20),
6	email VARCHAR(100),
7	direccion VARCHAR(200)
8	);
9	--tabla de mascotas
10	CREATE TABLE mascotas (
11	id_mascota SERIAL PRIMARY KEY,
12	nombre VARCHAR(100) NOT NULL,
13	especie VARCHAR(50) NOT NULL,
14	raza VARCHAR(50),
15	edad INT,
16	id_propietario INT NOT NULL,
17	
18	CONSTRAINT fk_mascotas_propietarios
19	FOREIGN KEY (id_propietario)
20	REFERENCES propietarios(id_propietario)
21	ON DELETE CASCADE
22	);
23	
24	--tabla veterinarios
25	CREATE TABLE veterinarios (
26	id_veterinario SERIAL PRIMARY KEY,
27	nombre VARCHAR(100) NOT NULL,
28	especialidad VARCHAR(100),
29	telefono VARCHAR(20)
30	);
31	
32	--tabla citas
33	CREATE TABLE citas (
34	id_cita SERIAL PRIMARY KEY,
35	id_mascota INT NOT NULL,
36	id_veterinario INT,
37	fecha TIMESTAMP NOT NULL,
38	estado VARCHAR(20) NOT NULL DEFAULT 'programada',
39	
40	CONSTRAINT fk_citas_mascotas
41	FOREIGN KEY (id_mascota)
42	REFERENCES mascotas(id_mascota)
43	ON DELETE RESTRICT,
44	
45	CONSTRAINT fk_citas_veterinarios
46	FOREIGN KEY (id_veterinario)
47	REFERENCES veterinarios(id_veterinario)
48	ON DELETE SET NULL
49	);
50	
51	--tabla historial de atenciones
52	CREATE TABLE historial_atenciones (
53	id_historial SERIAL PRIMARY KEY,
54	id_cita INT NOT NULL,
55	diagnostico TEXT DEFAULT 'Pendiente de detalle',
56	fecha_registro TIMESTAMP DEFAULT NOW(),
57	
58	CONSTRAINT fk_historial_citas
59	FOREIGN KEY (id_cita)
60	REFERENCES citas(id_cita)
61	ON DELETE CASCADE
62	);

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 159 msec.

-INSERCIÓN DE DATOS

```
1  --insertar datos en propietarios
2  INSERT INTO propietarios (nombre, telefono, email, direccion) VALUES
3  ('Maria Lopez', '0991234567', 'maria@gmail.com', 'Av. Ecuador 123'),
4  ('Carlos Perez', '0997654321', 'carlos@gmail.com', 'Calle Larga 456'),
5  ('Ana Ramirez', '0998887777', 'ana@gmail.com', 'Av. America 789');
6
7  --insertar datos en mascotas
8  INSERT INTO mascotas (nombre, especie, raza, edad, id_propietario) VALUES
9  ('Max', 'Perro', 'Labrador', 3, 1),
10 ('Luna', 'Gato', 'Persa', 2, 1),
11 ('Toby', 'Perro', 'Pastor Aleman', 5, 2),
12 ('Mimi', 'Gato', 'Siames', 1, 3);
13
14 --insertar datos en veterinarios
15 INSERT INTO veterinarios (nombre, especialidad, telefono) VALUES
16 ('Dr. Fernando Diaz', 'Cirugia', '0991112222'),
17 ('Dra. Carolina Rojas', 'General', '0993334444');
18
19 --insertar datos en citas
20 INSERT INTO citas (id_mascota, id_veterinario, fecha, estado) VALUES
21 (1, 1, '2026-07-15 09:00:00', 'programada'),
22 (2, 2, '2026-07-15 10:30:00', 'programada'),
23 (3, 1, '2026-07-14 14:00:00', 'atendida'),
24 (4, NULL, '2026-07-16 11:00:00', 'programada');
25
26 --insertar datos en historial-primero debe pasar la cita a esta completado
27 INSERT INTO historial_atenciones (id_cita, diagnostico) VALUES
28 (3, 'Vacunacion completada');
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 42 msec.

-JOIN

```
Query History
1  --2 join mascotas-propietarios
2  SELECT
3      m.nombre          AS mascota,
4      p.nombre          AS dueño,
5      v.nombre          AS veterinario,
6      c.fecha
7  FROM citas c
8  INNER JOIN mascotas m ON c.id_mascota = m.id_mascota
9  INNER JOIN propietarios p ON m.id_propietario = p.id_propietario
10 --mostrar solo citas en estado programada
11 LEFT JOIN veterinarios v ON c.id_veterinario = v.id_veterinario
12 WHERE c.estado = 'programada';
13
14
```

Data Output Messages Notifications

	mascota character varying (100) 🔒	dueño character varying (100) 🔒	veterinario character varying (100) 🔒	fecha timestamp without time zone 🔒
1	Max	Maria Lopez	Dr. Fernando Díaz	2026-07-15 09:00:00
2	Luna	Maria Lopez	Dra. Carolina Rojas	2026-07-15 10:30:00
3	Mimi	Ana Ramirez	[null]	2026-07-16 11:00:00

-VISTA

```
Query    Query History

1  --vista citas programadas-primero probar si el join es correcto
2  CREATE OR REPLACE VIEW vista_citas_programadas AS
3  SELECT
4      m.nombre          AS mascota,
5      p.nombre          AS dueño,
6      v.nombre          AS veterinario,
7      c.fecha
8  FROM citas c
9  INNER JOIN mascotas m ON c.id_mascota = m.id_mascota
10 INNER JOIN propietarios p ON m.id_propietario = p.id_propietario
11 LEFT JOIN veterinarios v ON c.id_veterinario = v.id_veterinario
12 WHERE c.estado = 'programada';
13
14 SELECT * FROM vista_citas_programadas;
```

Data Output Messages Notifications

	mascota character varying (100)	duenio character varying (100)	veterinario character varying (100)	fecha timestamp without time zone
1	Max	Maria Lopez	Dr. Fernando Diaz	2026-07-15 09:00:00
2	Luna	Maria Lopez	Dra. Carolina Rojas	2026-07-15 10:30:00
3	Mimi	Ana Ramirez	[null]	2026-07-16 11:00:00

-INDICE

The screenshot shows a SQL IDE with a query editor and a query plan output. The query is as follows:

```
1  --consulta sin indice
2  EXPLAIN ANALYZE
3  SELECT * FROM citas
4  WHERE id_mascota = 1 AND estado = 'programada';
5
```

The query plan output is shown in a table below the query editor:

	QUERY PLAN
1	Seq Scan on citas (cost=0.00..21.55 rows=1 width=78) (actual time=0.018..0.019 rows=1 loop...
2	Filter: ((id_mascota = 1) AND ((estado)::text = 'programada'::text))
3	Rows Removed by Filter: 3
4	Planning Time: 0.445 ms
5	Execution Time: 0.052 ms

La consulta sin índice muestra un tiempo de ejecución de 0.052 ms, tomando en cuenta que hay muy pocos datos, pero si fueran ciento o miles esto aumentaría considerablemente.

The screenshot shows a SQL IDE with a query editor and a query plan output. The query is as follows:

```
6  --creacion de indice
7  CREATE INDEX idx_citas_mascota_estado
8  ON citas (id_mascota, estado);
9
10 --consulta con indice
11 EXPLAIN ANALYZE
12 SELECT * FROM citas
13 WHERE id_mascota = 1 AND estado = 'programada';
14
15
```

The query plan output is shown in a table below the query editor:

	QUERY PLAN
1	Seq Scan on citas (cost=0.00..1.06 rows=1 width=78) (actual time=0.010..0.010 rows=1 loop...
2	Filter: ((id_mascota = 1) AND ((estado)::text = 'programada'::text))
3	Rows Removed by Filter: 3
4	Planning Time: 0.236 ms
5	Execution Time: 0.023 ms

Veamos que el tiempo de ejecución es de 0.023 ms una mejora considerable y tomando en cuenta lo antes mencionado cuando haya bastantes datos el tiempo de consulta se reduce a la mitad.

-PROCESO ALMACENADO

Query	Query History
1	--procedimienot para registrar cita
2	CREATE OR REPLACE PROCEDURE registrar_cita(
3	p_id_mascota INT,
4	p_id_veterinario INT,
5	p_fecha TIMESTAMP
6	)
7	LANGUAGE plpgsql
8	AS \$\$
9	DECLARE
10	--variables para guardar los resultados de las validaciones
11	mascota_existe INT;
12	vet_existe INT;
13	mensaje_error TEXT;
14	BEGIN
15	--verificar que la mascota exista
16	SELECT COUNT(*) INTO mascota_existe
17	FROM mascotas
18	WHERE id_mascota = p_id_mascota;
19	
20	IF mascota_existe = 0 THEN
21	RAISE EXCEPTION 'La mascota con id % no existe', p_id_mascota;
22	END IF;
23	
24	--verificar que el veterinario exista
25	SELECT COUNT(*) INTO vet_existe
26	FROM veterinarios
27	WHERE id_veterinario = p_id_veterinario;
28	
29	IF vet_existe = 0 THEN
30	RAISE EXCEPTION 'El veterinario con id % no existe', p_id_veterinario;
31	END IF;
32	
33	--si todo esta bien, insertar la cita
34	INSERT INTO citas (id_mascota, id_veterinario, fecha, estado)
35	VALUES (p_id_mascota, p_id_veterinario, p_fecha, 'programada');
36	
37	RAISE NOTICE 'Cita creada exitosamente para la mascota %', p_id_mascota;
38	
39	EXCEPTION
40	WHEN OTHERS THEN
41	--capturar el error en una variable intermedia
42	GET STACKED DIAGNOSTICS mensaje_error = MESSAGE_TEXT;
43	RAISE NOTICE 'Error al crear la cita: %', mensaje_error;
44	END;
45	\$\$;
46	
47	--ejecucion normal
48	CALL registrar_cita(1, 2, '2026-07-20 08:00:00');
49	
50	--ejecucion error mascota
51	CALL registrar_cita(99, 1, '2026-07-20 09:00:00');
52	
53	--ejecucion error veterinario
54	CALL registrar_cita(1, 99, '2026-07-20 10:00:00');

Data Output Messages Notifications

NOTICE: Cita creada exitosamente para la mascota 1
NOTICE: Error al crear la cita: La mascota con id 99 no existe
NOTICE: Error al crear la cita: El veterinario con id 99 no existe
CALL

Query returned successfully in 59 msec.

-TRIGGER

Query

Query History

```
1  --funcion trigger registrar historial
2  CREATE OR REPLACE FUNCTION fn_registrar_historial()
3  RETURNS TRIGGER
4  LANGUAGE plpgsql
5  AS $$
6  BEGIN
7  --solo crear historial si el nuevo estado es atendida
8  IF NEW.estado = 'atendida' THEN
9      INSERT INTO historial_atenciones (id_cita, diagnostico)
10     VALUES (NEW.id_cita, 'Pendiente de detalle');
11     END IF;
12
13     RETURN NEW;
14 END;
15 $$;
16
17 --ejecucion de trigger - cita cambia de estado
18 CREATE TRIGGER trg_registrar_historial
19 AFTER UPDATE OF estado
20 ON citas
21 FOR EACH ROW
22 EXECUTE FUNCTION fn_registrar_historial();
23
24 --cambiar una cita a estado atendido
25 UPDATE citas SET estado = 'atendida' WHERE id_cita = 1;
26
27 --verificar que se creo el historial
28 SELECT * FROM historial_atenciones;
29
```

Data Output

Messages

Notifications

	id_historial [PK] integer	id_cita integer	diagnostico text	fecha_registro timestamp without time zone
1	1	3	Vacunacion completa...	2026-07-13 23:20:23.021112
2	2	1	Pendiente de detalle	2026-07-13 23:59:08.876263

En la sección A creamos en apartado de Observer donde también hay clases que notifican por correo y sms cuando cambia el estado de una reserva. Se podría decir que este trigger creado cuando cambia el estado a atendida de igual manera se activa y registra en el historial de clientes atendidos funcionaria igual si quisiéramos crear otros para pendientes, o programadas. Considero que son lenguajes distintos C# y SQL pero para este procedimiento si se parecen.

ENLACE DEL REPOSITORIO EN GIT HUB:

https://github.com/JOALU26/EXAMEN_FUNDAMENTOS_II.git